

Humanoid Academy

Tag 4: Maschinelles Lernen
und Anwendungen



Zeitplan

Tag 4

Di 18.06.2026

08:30 Einfinden der Teilnehmer

09:00 Schulungsblock 1

10:30 Kaffee- und Teepause

10:45 Schulungsblock 2

12:00 Mittagessen

13:00 Schulungsblock 3

14:30 Kaffee- und Teepause mit Snacks

15:00 Schulungsblock 4

16:00 Ende Tag 4



Systemüberblick G1 (Tag 1 Recap)

G1 Überblick & Safety Hardware, Sensoren

- **Verbindung & Steuerung**
AP / Ethernet / SSH
- **Programmierung**
SDK2, FSM, ROS2
- **Entwicklungsumgebung**
Linux, Python, CycloneDDS
- **Tools & Demos**
Repository, CLI, Beispielskripte



Systemüberblick G1 (Tag 2)

Tag 2 –Loco Client and SLAM- Navigation



Systemüberblick G1 (3 Recap)

Tag 3 – Arm, Manipulation

- 3-Finger
- Greifsteuerung(mit 3 finger)



Tagesziele(use cases)

- RL (Introduktion)
- VLA (Introduktion)
- Perzeption
- VLA mit G1
- Aufgabe



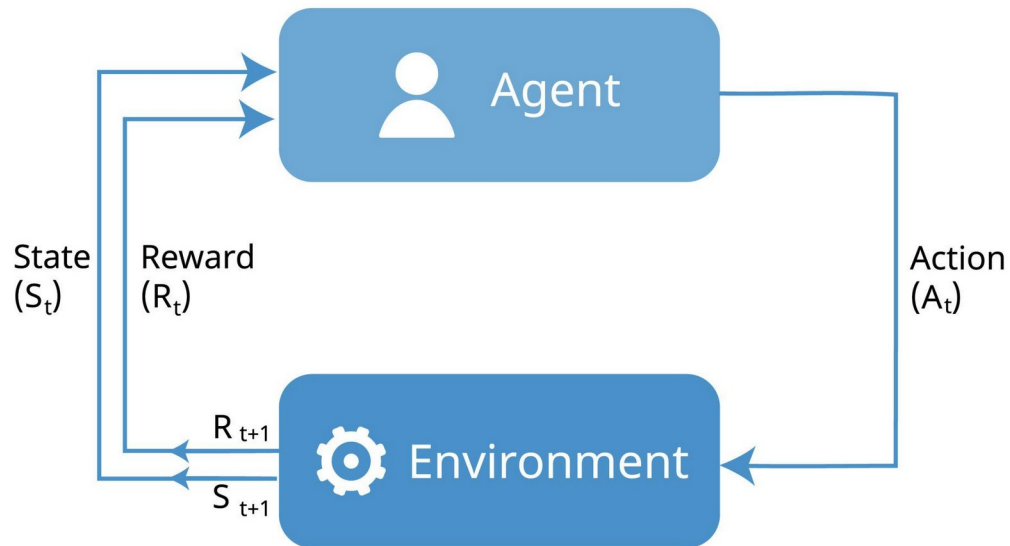
What is RL

- <https://www.youtube.com/watch?v=Kf9WDqYKYQQ&t=103s>
- Footstep Learning: <https://www.youtube.com/watch?v=U8gOafef5y8>



RIENFORCEMENT LEARNING

Reinforcement Learning



Reinforcement Learning = Learning from Reward

- **Agent** – the learner (e.g., a robot)
 - **Environment** – where the agent acts
 - **Actions** – things the agent can do
 - **States & Rewards** – feedback that teaches good behavior
 - **Policy** – what strategy the agent uses to choose actions
 - **Goal** – maximize cumulative reward
- a machine-learning paradigm where a learner interacts with an environment and improves by receiving rewards from its actions.

Example analogy: -reward good behavior, ignore bad behavior.

EXAMPLE

Teaching a Robot to Walk

What does the robot see?

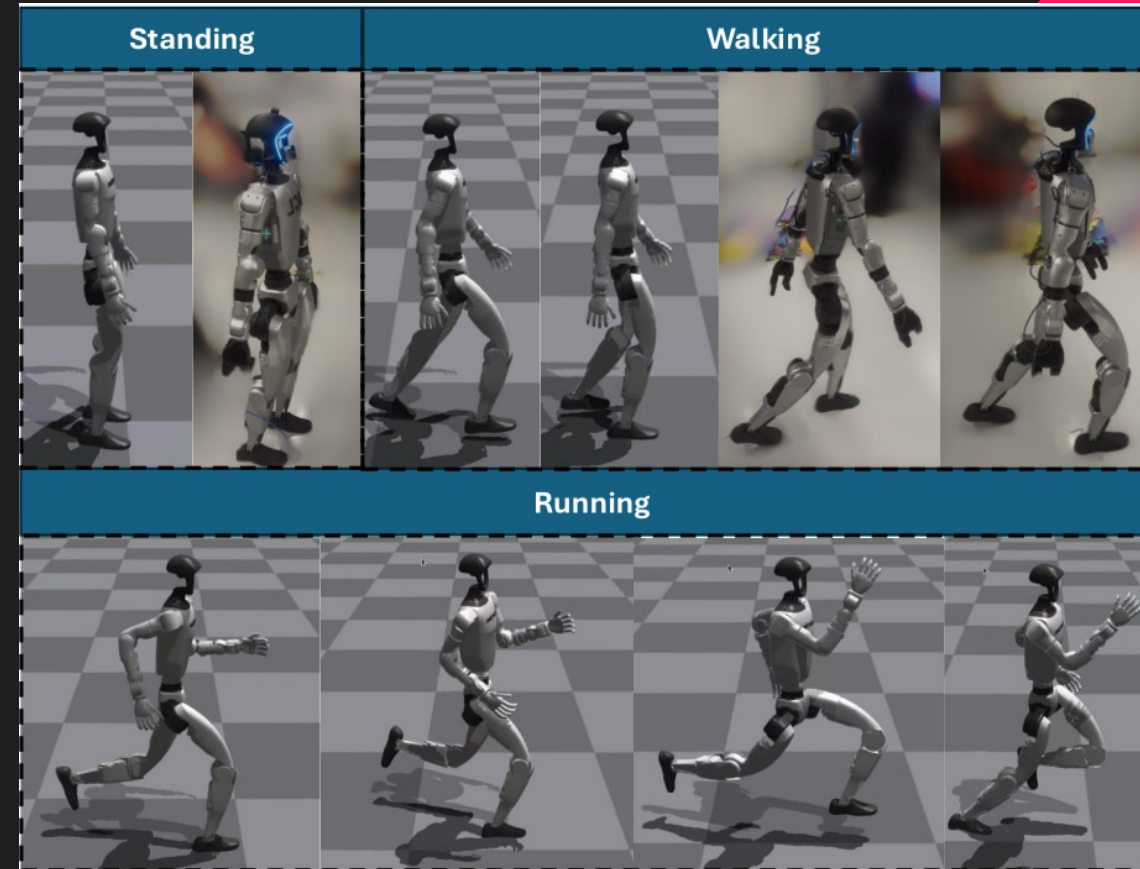
- Joint angles
- Balance (IMU)
- Speed

What does it control?

- Motor torques

What reward does it get?

- Move forward
- Stay upright
- Fall down



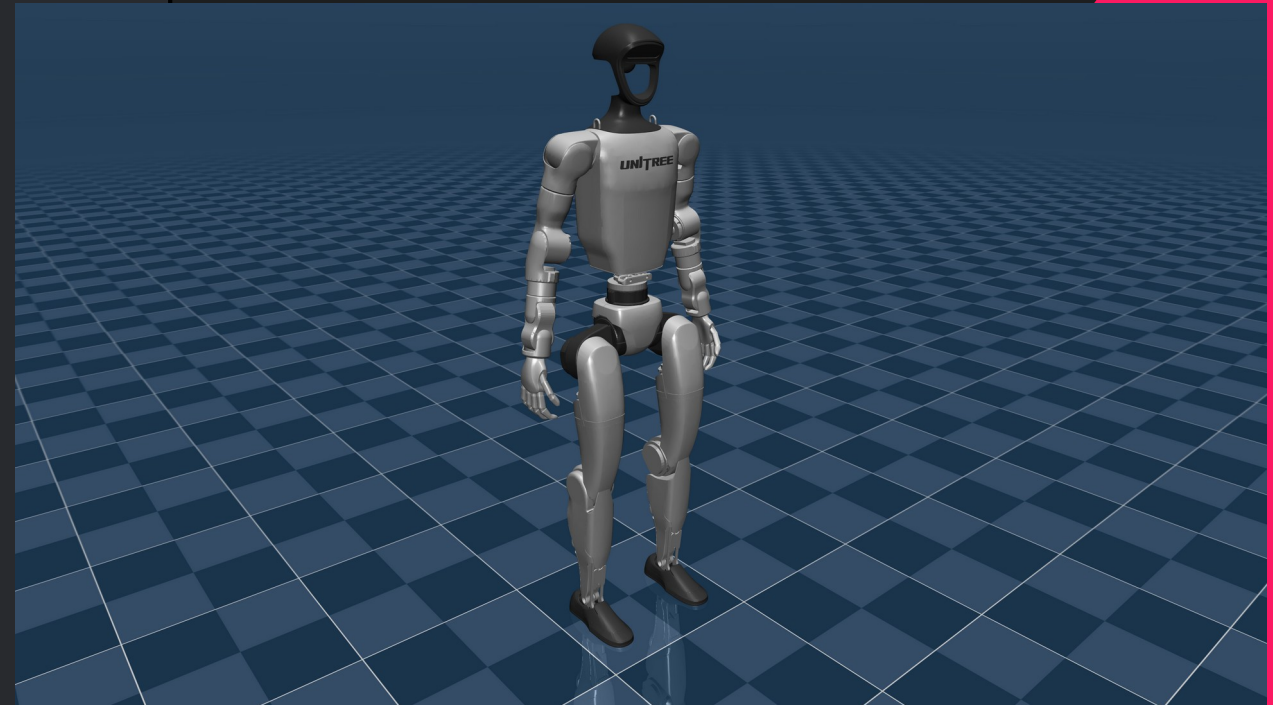


Training Happens in Simulation

If we trained on the real robot, it would fall “10,000 times.”

We use simulation:

- Safe
- Fast
- Cheap



How It Improves

Try random actions

1. Get reward
2. Adjust neural network
3. Try again
4. Over millions of tries → becomes stable.



REAL ROBOT EXAMPLE

- Simulation → Train → Copy brain to robot → Walk.



Why RL is Powerful

- Can learn walking
- Can learn jumping
- Can learn recovery
- Can adapt

**We don't tell it how to walk.
We tell it what we want.**



Step-by-Step Working Process

Step 1 – Observe

The robot reads sensors:

- Joint positions
- Joint velocities
- IMU (tilt, balance)
- Target direction
- This is called the **state**.



Step 2 – Take Action

- A neural network (the "brain") decides:
- How much torque to send to each motor
- That is the **action**.



Step 3 – Get Reward

After moving:

- If robot stays upright → + reward
- If robot moves forward → + reward
- If robot falls → big penalty
- This is the **reward**.



Step 4 – Improve the Brain

The learning algorithm checks:

- “Did this action increase reward?”
- If yes → make this behavior stronger.
- If no → adjust network.



Step 5 – Repeat Millions of Times

The robot:

1. Tries
2. Fails
3. Adjusts
4. Tries again

- After millions of steps → stable walking.



Tools Used in Reinforcement Learning for Robots

Common Simulators:

- MuJoCo
- Isaac Gym
- PyBullet



Machine Learning Frameworks

To train neural networks:

- PyTorch
- TensorFlow
- Updates the neural network



Py Torch Vs TensorFlow

Framework	Best For	Difficulty
PyTorch	Robotics, RL, VLA, Research	Easy
TensorFlow	Production Systems	Medium

RL Algorithms

- PPO (Proximal Policy Optimization)
 - SAC(Soft Actor Critic)
 - DDPG(Deep Deterministic Policy Gradient)
-
- PPO is stable and widely used for walking robots.

PPO	Small safe updates	★★★★★	Walking robots
SAC	Smart + explores	★★★★★	Advanced control
DDPG	Direct precise control	★★	Continuous torque tasks

Algorithm	Full Form	Main Idea	Pros	Cons
PPO	Proximal Policy Optimization	Small, safe policy updates	Stable, easy to train	Needs lots of data
SAC	Soft Actor-Critic	Learns while exploring many actions	Very sample efficient	More complex
DDPG	Deep Deterministic Policy Gradient	Directly outputs continuous actions	Fast in simple tasks	Unstable, sensitive
Q-Learning/DQN	Deep Q Network	Learns value of actions	Good for games	Doesn't work well for many robot joints

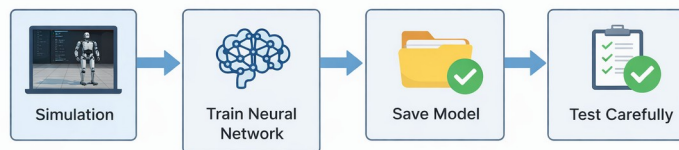
Robot Middleware

- ROS / ROS2
- Custom SDK (like Unitree SDK)



Hardware Needed

- GPU (for training)
- Real robot platform
- Sensors (IMU, encoders)

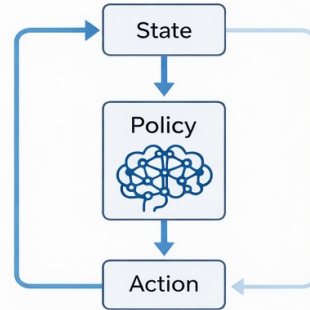


•NVIDIA GeForce RTX 3060 / 3080 / 4090

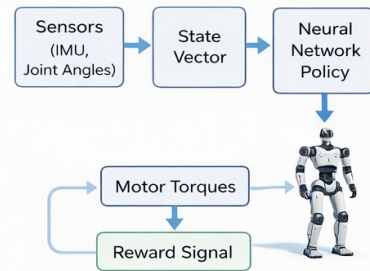
•Or NVIDIA A100 (data center)



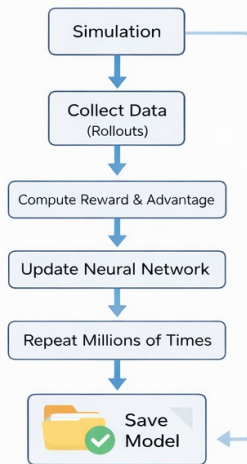
1 Core Reinforcement Learning Loop



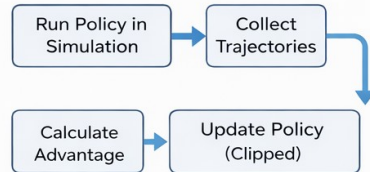
2 Humanoid Walking Control Flow



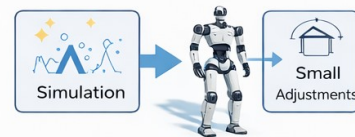
3 Simulation to Real Deployment Pipeline



4 Proximal Policy Optimization (PPO)



5 Simulation to Real Robustness Flow



VLA ??

- <https://youtu.be/m1CH-mgpdYg?si=OYEvfjEZlexIZDpO>
- https://research.nvidia.com/labs/gear/gr00t-n1_5/



Vision-Language-Action (VLA)

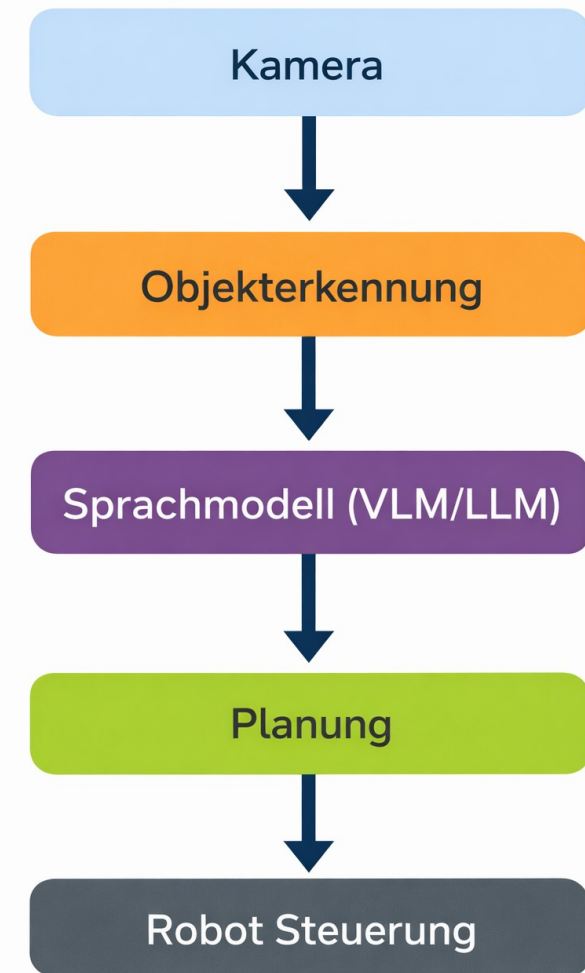
- Vision → Wahrnehmung (Kamera)
- Language → Verständnis (Befehl)
- Action → Ausführung (Roboter)

Beispiel:

"Pick the box" → Objekt erkennen → Greifen



VLA Pipeline



Wahrnehmung (Perception)

- RGB / Depth Kamera
- Objekterkennung
- Bounding Box / Segmentierung



Sprachverarbeitung g

- Eingabe: "Pick the bottle"
- Modell interpretiert Aufgabe
- Generiert Aktionen



VLA vs RL

VLA vs RL

RL

Learns from rewards

Used for walking and control

Training takes millions of trials

Focus on motor skills

VLA

Learns from images and language

Used for task understanding

Uses large pretrained models

Focus on decision making

KI Integration

Kostenlose Sprachmodelle

- Huggingface
 - <https://huggingface.co/>
- Ollama
 - <https://ollama.com/search>



Aufgabe 1

- Chat Modus erstellen
 - Mikrophonzugriff (wake up mode)
 - Ros2 topic echo /audio_msg
 - Verknüpfung mit dem ollama qwen3.5:9b or gemma3:1b Sprachmodell (durch codex)
 - Das ollama modell läuft über 127.0.0.1:11434



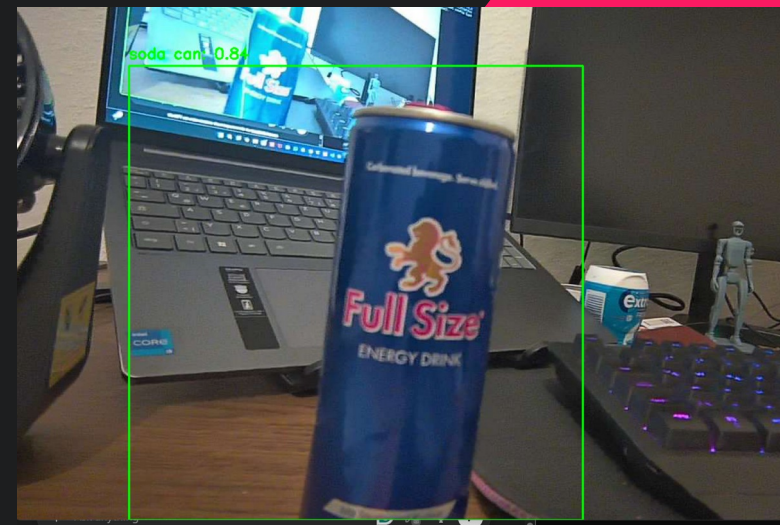
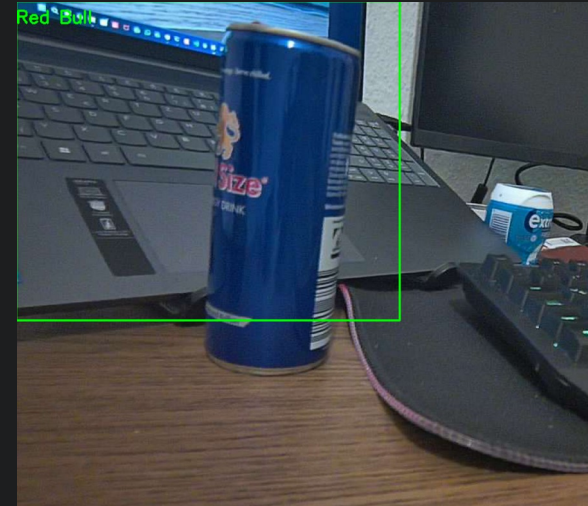
Aufgabe 2

- Chat modus mit Kenntnisse erweitern
 - Mit codex einen RAG Ansatz zur Ergänzung vom Chatbot mit der verfügbaren `robot_modules_knowledge.sample.json` Datei



Objekterkennung

- **Modell: OpenAI CLIP**
 - (Contrastive Language-Image Pretraining)
- **Required Setup**
 - Python 3.8+
 - PyTorch
 - OpenCV / Pillow
 - OpenAI CLIP model
 - GPU empfohlen für schnellere Inferenz



Zugriff von Kameradaten

- **VideoClient -> RGB Daten**
- Sdk_client:
 - Robot.rgbd_host = „10.34.0.206“
 - Robot.get_rgbd()

Aufgabe 3

- Codex den perception_clip.ipynb notebook durchlesen lassen
- Mit codex ein Skript erstellen, wobei der G1 schrittweise für 2 Sekunden nach vorne läuft, bis ein Objekt (blauer Sitzsack) erkannt ist. Dann stoppen



Herausforderungen

- Sensorrauschen
- Falsche Erkennung
- Navigationsprobleme



Bonusaufgabe

- Codex den perception_clip.ipynb notebook durchlesen lassen
- Chatbot ergänzen zur Beschreibung der Umgebung



Lernergebnis

- Verständnis der VLA Pipeline
- Verbindung Vision → Action
- Autonome Robotersteuerung

